实 验 报 告

实验名称：实验八 BP 算法分析与实现

课程名称：认知科学与类脑识别

实验地点：K2

实验日期：2024.10.25

班 级：2022 级人工智能班

姓 名：薛润泽

学 号：202200130077

一 实验目的：

加深对 BP 算法的理解，能够使用 BP 算法解决简单问题

二 实验环境：

Windows

三 实验内容：

根据 BP 算法的相关知识，使用 Python 语言实现一个简单 BP 算法模 型。

实验要求：

（1） 输入数据[[0.35], [0.9]]，真实值[[0.5]]，输出数据与真 实值的误差使用平方和误差表示。

（2） 创建人工神经网络。

（3） 参数更新值 Δw 用链式求导法则求出，最后使得输出数据与真 实值的误差小于 0.01

四 实验步骤：

1、实验程序：

实现梯度的反向传播需要构建一个简单的神经网络，在本实验中构建出的简单神经网络包括一个输入层、一个隐藏层和一个输出层。

（1）输入数据 input = [[0.35]， [0.9]]， targets = [[0.5]]，并设定期望的误差阈值 error_threshold（当训练过程前向计

算时，如果预测值和真实值的误差已经小于该值，可以停止训练），以及学习率 learning_rate。

ex8.py > sigmoid

```
import numpy as np

# 输入数据，两个样本，每个样本一个特征
inputs = np.array([[0.35], [0.9]])
# 真实值
targets = np.array([[0.5]])
# 期望的误差阈值
error_threshold = 0.01
# 学习率
learning_rate = 0.1

```

（2）初始化网络参数，因为预计构建三层神经元，所以会有两层层间权重矩阵 w 和偏置 b，将这两层的 w 和 b 进行初始化。

weights_input_hidden 是输入层到隐藏层的权重矩阵。

weights_hidden_output 是隐藏层到输出层的权重矩阵。

bias_hidden 是隐藏层的偏置。

bias_output 是输出层的偏置。

```

# 初始化网络参数
# 假设隐藏层有3个神经元，每个样本有一个输入特征
weights_input_hidden = np.random.randn(1, 3)
weights_hidden_output = np.random.randn(3, 1)
bias_hidden = np.random.randn(3)
bias_output = np.random.randn(1)  # 保持为一维数组

```

（3）设置 Sigmoid 函数，并计算 Sigmoid 函数对于 x 求导之

后的导数

```
18
19  def sigmoid(x):
20      return 1 / (1 + np.exp(-x))
21
22  def sigmoid_derivative(x):
23      return x * (1 - x)
24
```

（4）设置网络训练次数，并计算每一次前向传播的值

```
25  # 训练网络
26  max_epochs = 10000
27  for epoch in range(max_epochs):
28      # 前向传播
29      hidden_layer_input = np.dot(inputs, weights_input_hidden) + bias_hidden
30      hidden_layer_output = sigmoid(hidden_layer_input)
31
32      output_layer_input = np.dot(hidden_layer_output, weights_hidden_output) + bias_output
33      predicted_output = sigmoid(output_layer_input)
```

（5）计算前向传播的预测值与真实值的误差，并完成梯度反向传播。

```
34
35      # 计算误差
36      error = targets - predicted_output
37
38      # 反向传播
39      output_error_term = error * sigmoid_derivative(predicted_output)
40      hidden_error_term = np.dot(output_error_term, weights_hidden_output.T) * sigmoid_derivative(hidden_layer_output)
41
```

（6）梯度反向传播后，更新两层权重矩阵 w 的值以及两个偏置 b 的值。

```
# 更新权重和偏置
weights_hidden_output += np.dot(hidden_layer_output.T, output_error_term) * learning_rate
bias_output += np.sum(output_error_term, axis=0) * learning_rate

weights_input_hidden += np.dot(inputs.T, hidden_error_term) * learning_rate
bias_hidden += np.sum(hidden_error_term, axis=0) * learning_rate
```

（7）每进行 1000 次训练输出一次损失函数值，观察训练情况。

```
    if epoch % 1000 == 0:
        loss = np.mean(np.square(error))
        print(f'Epoch {epoch}, Loss: {loss}')
```

（8）最后结果存在 predicted_output 中

```
52
53  print("训练完成")
54  print("最终预测输出:", predicted_output)
```

2、程序实验结果：

进行 10000 次训练，结果如下。

```
PS D:\大学\大三上\认知科学与类脑识别\ex8> python -u "d:\大学\大三上\认知
Epoch 0, Loss: 0.03309704414168001
Epoch 1000, Loss: 9.825797609480395e-05
Epoch 2000, Loss: 3.398181965758631e-05
Epoch 3000, Loss: 1.1783119820107373e-05
Epoch 4000, Loss: 4.092484689971188e-06
Epoch 5000, Loss: 1.422819687315284e-06
Epoch 6000, Loss: 4.94965822698345e-07
Epoch 7000, Loss: 1.7224922862103384e-07
Epoch 8000, Loss: 5.995597416608874e-08
Epoch 9000, Loss: 2.0871938969024846e-08
训练完成
最终预测输出: [[0.49991101]
 [0.50008142]]
PS D:\大学\大三上\认知科学与类脑识别\ex8>
```

即，我们输入[[0.35], [0.9]]后，希望网络能将输入值转化为 target 值，最终网络将两个输入值转化为[0.49991101]和[0.50008142]。

3、分析和改进：

通过分析输入值域输出值的差距，发现差距小于 0.0001，网络实现效果好。

但是网络输入数据过少，不能清楚知道网络在面对较多样本数据时的输出效果。

五 实验小结：

反向传播算法（Backpropagation）是训练神经网络的关键算法，用于计算梯度并更新网络的权重和偏置，以最小化损失函数。本实验通过对简单样本构建简单神经网络，并应用反向传播算法，使我了解到如何使用链式法则来计算梯度反向传播，从而实现参数更新。